A Web Technologies LAB (9FC66) Project Report

On

Smart Portfolio Management System

B.Tech COMPUTER SCIENCE & ENGINEERING, 2025-2026

Submitted By	Batch Number: E02
k varsha	23311A05v3
A Maheshwari	23311A05w4
Depa Urvi	23311A05W7
J Laxmi varshitha	23311A05X0
M Karthik Reddy	23311A05Z3
P Jeevana Jyothi	23311A05AA

Instructor: Dr.Dheeraj Sundaragiri

Department of Computer Science and Engineering

SREENIDHI INSTITUTE OF SCIENCE AND TECHNOLOGY (AUTONOMOUS)

Yamnampet (V), Ghatkesar (M), Hyderabad - 501301, T.S.

ABSTRACT

ReachZone is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js) with Vite as the frontend build tool, designed to help college students craft professional cold outreach emails for internships, referrals, and mentorship opportunities. The application integrates the OpenAI API to generate personalized, context-aware emails based on student inputs such as name, target role, company, and purpose. Since direct email sending involves complex infrastructure, ReachZone adopts a simple copy-and-send approach. It additionally features pre-built email templates and an Interview Prep Guide covering HR, Technical, and Behavioral categories. The backend is built with Express.js on Node.js with modular routing, JWT-based authentication, and MongoDB Atlas as the cloud database managed through Mongoose. ReachZone demonstrates a practical, real-world implementation of modern web technologies, addressing a genuine communication gap faced by students in their career journey.

Tools and Technologies Used

- **Frontend:** React.js, HTML5, CSS3, JavaScript (ES6+), npm/yarn
- **Backend:** Node.js, Express.js, JavaScript (ES6+), npm/yarn
- **Database:** MongoDB, Mongoose (ODM for Node.js)
- **Development Environment:** Visual Studio Code, Git, Postman (for API testing)
- **Deployment (Optional):** Heroku, Netlify, Vercel, AWS, Render

Nature of the Project

Batch No.	**Title**	**Nature of the Project**	**Domain of the Project**
B01	Reach Zone	Application	Web Technologies & Full-Stack Development
		Prototype	

Note: Please tick ✓ Appropriate nature of the project (Application or Prototype).

Table 1: Correlation of Project with POs/PSOs

ID	**Program Outcomes / Program Specific Outcomes**	**Correlation**
PO1	Engineering Knowledge	
PO2	Problem analysis	
PO3	Design/development of solutions	
PO4	Conduct investigations of complex problems	
PO5	Modern tool usage	
PO6	The engineer and society	
PO7	Environment and sustainability	
PO8	Ethics	
PO9	Individual and team work	
PO10	Communication	
PO11	Project management and finance	
PO12	Life-long learning	
PSO1	Core courses and modern tools	
PSO2	Diverse and emerging areas	
PSO3	Seminars and projects	

Note: Level of Correlation: H: High, M: Moderate, L: Low

Table 2: Mapping of Project with Sustainable Development Goals (SDGs)

SDG #	**Sustainable Development Goal**	**Relevance**
SDG 1	No Poverty	
SDG 2	Zero Hunger	
SDG 3	Good Health and Well-being	
SDG 4	Quality Education	
SDG 5	Gender Equality	
SDG 6	Clean Water and Sanitation	
SDG 7	Affordable and Clean Energy	
SDG 8	Decent Work and Economic Growth	
SDG 9	Industry, Innovation and Infrastructure	
SDG 10	Reduced Inequalities	
SDG 11	Sustainable Cities and Communities	
SDG 12	Responsible Consumption and Production	
SDG 13	Climate Action	
SDG 14	Life Below Water	
SDG 15	Life on Land	
SDG 16	Peace, Justice and Strong Institutions	
SDG 17	Partnerships for the Goals	

Note: Please tick ✓ to the appropriate SDGs.

Contents

1 Introduction **4**

2 Project Objectives **4**

3 System Architecture **4**
3.1 Client-Side (React) 5
3.2 Server-Side (Node.js & Express.js) 5
3.3 Database (MongoDB) 5

4 Deployment Architecture **5**

5 Project Design and Implementation **5**
5.1 Database Schema Design 5
5.2 API Endpoints 5
5.3 Frontend Components 6
5.4 Code 7
5.4.1 Backend API (index.js) 7
5.4.2 Frontend Component (AuthContext.jsx) 7

6 Testing **8**
6.1 API Testing 8

7 Results and Discussion **9**

8 Conclusion **9**

9 References **9**

1 Introduction

ReachZone is a MERN stack (MongoDB, Express.js, React.js, Node.js) web application that helps college students create professional cold outreach emails for internships, referrals, and mentorship. The frontend uses React.js with Vite for fast performance, while the backend uses Express.js with JWT-based authentication and OpenAI API integration to generate personalized emails. Instead of sending emails directly, it follows a copy-and-send approach. It also includes an Interview Prep Guide. MongoDB Atlas with Mongoose is used for the database, and sensitive credentials are managed securely using .env files.

2 Project Objectives

The main objectives of this project are:

- To design and develop a full-stack web application that provides an interactive and user-friendly experience.
- To implement secure user authentication and authorization, ensuring protected access to features like dashboard, profile, and other modules.
- To build dynamic functionalities such as content generation, roasting, and preparation modules that enhance user engagement.
- To create a responsive frontend interface using modern frameworks, ensuring compatibility across devices.
- To develop a robust backend system for handling APIs, user data, and application logic efficiently.
- To integrate a database for storing and managing user information securely.
- To deploy the frontend using Vercel for fast and optimized performance.
- To deploy the backend using Render for reliable server hosting and scalability.
- To ensure scalability and maintainability of the application for future enhancements like AI features or analytics.
- To gain practical experience in MERN stack development, deployment, and real-world project building.

3 System Architecture

The project follows the MERN stack architecture, which is a three-tier architecture consisting of the client-side, server-side, and database. This architecture ensures scalability, maintainability, and efficient data flow between components.

3.1 Client-Side (React)

The client-side is developed using React.js, which is used to build interactive and responsive user interfaces. It handles user interactions and communicates with the backend through HTTP requests (API calls). Routing is managed to navigate between different pages such as Home, Login, Dashboard, and Profile.

3.2 Server-Side (Node.js & Express.js)

The server-side is built using Node.js and Express.js. It is responsible for handling API requests, implementing business logic, managing authentication, and processing client data. The server acts as a bridge between the frontend and the database.

3.3 Database (MongoDB)

The application uses MongoDB as the database. It stores user data, application data, and other necessary information in a flexible, JSON-like format. MongoDB provides high scalability and efficient data management.

4 Deployment Architecture

The application is deployed using cloud platform:

- The Frontend is hosted on Vercel for fast and optimized delivery.
- The Backend is hosted on Render to ensure reliable API Handling and server performance.

5 Project Design and Implementation

5.1 Database Schema Design

Describe your MongoDB schema here. You can use a table or a more descriptive format.

Collection Name	Fields	Data Type
users	name	String
	email	String
	password	String
emails	recipientname	String
	recipientRole	String
	recipientCompany	String
interviewprep	fileName	String
	jobDescription	String

5.2 API Endpoints

- `POST /api/auth/register` - User Registration
- `POST /api/auth/login` - User Login

- `GET /api/templates` - Fetch All Templates
- `POST /api/templates/seed` - Seed Templates (Run Once)
- `GET /api/auth/profile` - Get User Profile (Protected Route)
- `PUT /api/auth/profile` - Update User Profile (Protected Route)
- `POST /api/emails/generate` - Generate Cold Outreach Email (Protected Route)
- `POST /api/emails/roast` - Roast Email Draft (Protected Route)
- `GET /api/emails` - Fetch Past Emails (Protected Route)
- `POST /api/emails/{MONGO_ID}/followup` - Generate Follow-Up Email (Protected Route)
- `POST /api/prep/generate` - Generate Interview Prep (Protected Route)
- `GET /api/prep` - Fetch Past Interview Preps (Protected Route)
- `GET /` - Load Home Page
- `GET /login` - Load Login Page
- `GET /register` - Load Register Page
- `GET /generate` - Access Generator Page (Protected Route)
- `GET /dashboard` - Access Dashboard Page (Protected Route)
- `GET /profile` - Access User Profile Page (Protected Route)
- `GET /roast` - Access Roast Feature Page (Protected Route)
- `GET /prep` - Access Interview Prep Page (Protected Route)

5.3 Frontend Components

- `Navbar.jsx`: Navigation bar with links to all pages; handles routing and conditionally shows options based on login state.
- `Home.jsx`: Landing page introducing ReachZone and its features.
- `Login.jsx` / `Register.jsx`: Authentication forms for user login and registration.
- `Generator.jsx`: Core email generation page; collects student inputs and displays the AI-generated email with a copy button.
- `Dashboard.jsx`: Shows the user's previously generated emails and saved history.
- `Profile.jsx`: Displays and manages user profile information.
- `Prep.jsx`: Interview Prep Guide page listing questions by category (HR, Technical, Behavioral).

- `Roast.jsx`: A feature that critiques a student's existing email draft using AI.
- `PrivateRoute.jsx`: Wrapper component that restricts access to authenticated users only.
- `AuthContext.jsx`: React context provider managing global authentication state across the app.

5.4 Code

5.4.1 Backend API (index.js)

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const { getMongoUri } = require('./mongoUri');
require('dotenv').config();

const app = express();
app.use(cors({
  origin: [
    "http://localhost:5173",
    "https://reach-zone-7jz83gop2-jeevanapalreddy-4080s-projects.vercel.app",
    /\.vercel\.app$/   // allows ALL vercel preview URLs of your project
  ],
  credentials: true
}));
app.use(express.json());

mongoose
  .connect(getMongoUri())
  .then(() => console.log('MongoDB connected'))
  .catch((err) => console.error('DB Error:', err));

app.use('/api/auth', require('./routes/auth'));
app.use('/api/emails', require('./routes/emails'));
app.use('/api/templates', require('./routes/templates'));
app.use('/api/prep', require('./routes/prep'));

const PORT = process.env.PORT || 5003;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Listing 1: Example index.js Route

5.4.2 Frontend Component (AuthContext.jsx)

```
import { createContext, useContext, useEffect, useState } from 'react';
import API from '../api';

export const AuthContext = createContext();

export function AuthProvider({ children }) {
```

```
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (!token) {
      setLoading(false);
      return;
    }

    API.get('/auth/profile')
      .then((res) => setUser(res.data))
      .catch(() => localStorage.removeItem('token'))
      .finally(() => setLoading(false));
  }, []);

  const login = async (email, password) => {
    const res = await API.post('/auth/login', { email, password });
    localStorage.setItem('token', res.data.token);
    setUser(res.data.user);
  };

  const register = async (data) => {
    const res = await API.post('/auth/register', data);
    localStorage.setItem('token', res.data.token);
    setUser(res.data.user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, register, logout,
   loading }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

Listing 2: Example React Component (AuthContext.jsx)

6 Testing

6.1 API Testing

API testing was performed using Postman. Various endpoints such as user authentication, data retrieval, and updates were tested to verify correct request and response behavior.

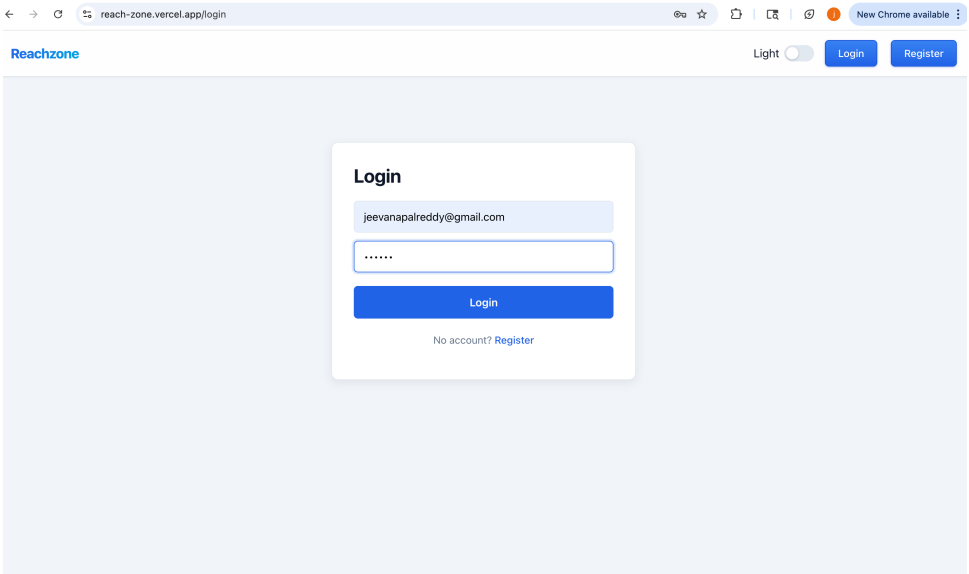

Figure 1: Screenshot of the Login Page

7 Results and Discussion

8 Conclusion

ReachZone demonstrates a complete, real-world MERN stack application integrated with OpenAI. It addresses a genuine need for college students — professional communication — and delivers it through a simple, usable interface. The project reinforces REST API design, React state management, MongoDB schema design, and third-party API integration, making it a comprehensive Web Technologies Lab deliverable.

9 References

List any external resources, tutorials, or documentation used during the project development.

- Node.js Documentation: https://nodejs.org/docs/
- Express.js Documentation: https://expressjs.com/
- React Documentation: https://react.dev/docs/
- MongoDB Documentation: https://www.mongodb.com/docs/
- Mongoose Documentation: https://mongoosejs.com/docs/

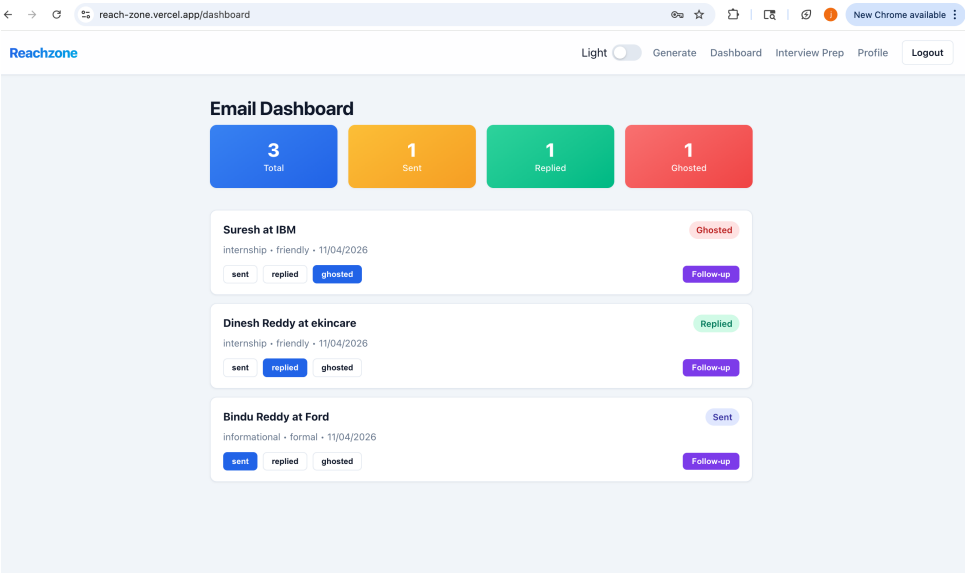

Figure 2: Screenshot of the User Dashboard